JHQ-GPU: Representation-Driven GPU Execution and Workload-Adaptive Hierarchical Refinement

Rui Luo and
Mengxuan Zhang

Australian National University, Canberra, Australia
Rui.Luo1@anu.edu.au, mengxuan.zhang@anu.edu.au

Abstract. JHQ scores approximate nearest neighbours in two stages: a compact primary code over many candidates, then a per-dimension residual code over the few survivors. Porting it to a GPU, we find the primary scan is not governed by how many bytes it moves. Six controls say so directly. A table-free form holds the least state of any variant and is among the slowest; two variants storing an identical number of entries diverge by up to 61%; and the granularity issuing the fewest instructions per candidate–subspace is slower than all of them, because its table does not stay resident. What the scan competes for is residency, and JHQ can win that competition in a way an ordinary product quantiser cannot: its Cartesian codebook makes the 256-entry primary table separate *exactly* into two 16-entry halves. For the second stage we choose the refinement budget with a label-free rule — compare the top-k sets the system returns at a candidate budget against a generous reference, on sampled workload queries — worth 1.095–1.469× in steady state. Its risk is the sample: at a quarter of the size we originally proposed, the quality loss varies enough by workload that we withdraw our own default. On six datasets to 3,072 dimensions and 17.8M vectors, JHQ is fastest in 18 of the 24 cells it shares with the other IVF-routed methods and indexes two where the tested IVF-RaBitQ build path runs out of memory, while the graph baseline CAGRA is 1.6–2.9× faster below Recall@10=0.95 at several times the build cost — a bounded operating region, not a universal-fastest claim.

Keywords: Approximate nearest neighbour search · GPU · Quantisation · High-dimensional data · Adaptive query processing

1 Introduction

Compressed nearest-neighbour search on a GPU is not governed by footprint: a representation can reduce bytes and still raise lookup pressure or instruction issue, and an implementation can move the same logical bytes and accelerate. JHQ [3] makes that concrete, because its hierarchy exposes two costs of different shape. After IVF routing a compact primary code scores every routed candidate at $O(n_c M)$; a per-dimension residual code is then read only for the c_k survivors

of primary filtering, at $O(c_k d)$ with $c_k \ll n_c$. JHQ also leaves the refinement factor α in $c_k = \alpha k$ externally specified. The port therefore raises two coupled questions with different causes: *how should the primary representation execute*, and *how much of the expensive hierarchy should execute*?

Our answer to the first begins from prior art rather than against it. Grouping code bits into small tables and choosing the group size by trading on-chip capacity against arithmetic is established — GPU IVF-RaBitQ [8] does exactly this, at U=4 — and we claim none of it. Two things are not established. That JHQ admits the grouping at all: its 8-bit code indexes 256 codewords with no grouped structure on its face, and only because each codeword is a Cartesian product of one-dimensional levels does the table separate exactly into two 16-entry halves, an identity an ordinary k-means product quantiser does not have. And which cost each grouping pays: full-table state grows as $256M$ entries against $32M$ for the split, but each further split costs another lookup per candidate–subspace, and which term dominates is what we measure rather than assert. Our answer to the second is a label-free workload policy: sample queries from the running workload, compare the top-k sets they return at a candidate budget against a generous reference, and take the smallest budget that leaves them unchanged. It needs neither ground-truth neighbours nor a learned difficulty model, at the cost of assuming a reasonably stable query distribution.

We evaluate on six datasets from 768 to 3,072 dimensions and up to 17.8M vectors, and we test the alternatives we rejected rather than only the implementation we kept. Six controls contradict the byte-count prediction, most sharply between two splits that hold the same number of entries and differ only in how many lookups reach them; static disassembly then supplies the implicated variable without a profiler, since the one granularity whose table does not fit issues the fewest instructions of any and is slower regardless. The budget rule is worth 1.095–1.469× at S=128, while at a quarter of that sample the quality risk varies enough by workload that the default we started from does not survive its own test.

The competitive picture is bounded and we state it as such. Among IVF-routed methods — the family JHQ belongs to, sharing its routing structure and interface — JHQ is fastest in 18 of the 24 comparable cells and indexes two datasets where the tested cuVS IVF-RaBitQ build path runs out of memory. It is not the fastest system measured: the graph baseline, CAGRA, is faster wherever the recall target is not high, by 1.6–2.9× below Recall@10=0.95 in its int8 form, at 3.9–7.4× the build time. We report that comparison in its own section rather than leave it out of the frontier without saying so.

Contributions. (i) Evidence that the primary scan's cost divides into an issue term and a residency term. **(ii)** An exact Cartesian factorisation of JHQ's primary distance table, derived from separability the codebook already has, with the regime in which it pays. **(iii)** A label-free policy for JHQ's residual budget, evaluated against fixed budgets and a ground-truth reference. **(iv)** An evaluation that reports where the compared systems are faster than ours.

2 Preliminaries

For a database vector x and query q in $\mathbb{R}^d$, JHQ applies an orthogonal $Q \in \mathbb{R}^{d\times d}$: $y = Qx$, $q' = Qq$ (row-major batches use $Y = XQ^\top$). Q is square, so this is a change of coordinates preserving Euclidean distance and hence the neighbour ordering, not dimensionality reduction. The transformed vector is split into M subspaces of dimension D_s, $d = MD_s$; each primary subspace code has B bits selecting one of $K = 2^B$ codewords, and each codeword is a Cartesian product of D_s one-dimensional Lloyd–Max levels. With L levels per coordinate $L = 2^{B/D_s}$, so the implementation admits only D_s dividing B: at B=8, $D_s \in \{1, 2, 4, 8\}$ gives $\{8, 4, 2, 1\}$ primary bits per coordinate. We use D_s=8, so each coordinate carries one primary bit and each subspace is one byte. This Cartesian construction is what Section 3.2 exploits.

The residual is $r = y - \hat{y}_{\text{primary}}$, the displacement from the primary reconstruction — *not* from an IVF centroid. IVF is an external routing layer that restricts the candidate set; it does not redefine what the residual quantiser represents. The residual code is per dimension, so at B_r bits it costs about dB_r bits per vector, and at B_r=8, D_s=8 the logical representation is about 1 primary plus 8 residual bits per dimension.

Routing yields a candidate set $C(q)$. The primary score D_P is evaluated for every routed candidate; JHQ keeps the c_k with the smallest primary scores and evaluates the hierarchical score D_H only for those, returning the top-k under D_H among them. We write $c_k = \alpha k$ for exposition; the implementation uses $c_k = \max(k, \lceil \alpha k \rceil)$, and since every evaluated $\alpha \geq 1$ the max only enforces the k-survivor floor. So α controls an irreversible filtering point: a neighbour dropped by primary ranking cannot be recovered by refinement, while routing can drop true neighbours before the primary stage at all. The two loss sources stay separate throughout.

3 GPU-Native JHQ

3.1 Execution and representation

We begin from the work JHQ requires after routing rather than from CUDA primitives. The primary stage is $O(n_c M)$ over compact codes and the residual stage $O(c_k d)$ over per-dimension codes, so three requirements follow: make primary access match candidate-parallel warp execution, keep the primary distance representation resident without replacing JHQ's quantiser, and preserve selective residual access instead of fusing accurate scoring into the broad scan. The orthogonal transform, primary codebook, residual definition and hierarchical ordering remain JHQ's; physical representation, execution and the external refinement policy change, and the scan uses a bounded per-thread top-αk selector whose deviation from exact selection Section 4.2 measures.

During the broad scan, warp lanes score different candidates while advancing through the same subspace m, so candidate-major $[N, M]$ storage has the warp read one byte per lane with stride M. We store the scan copy subspace-major,

$[M, N]$, which makes those 32 bytes adjacent: the transpose aligns the contiguous memory dimension with the dimension the warp shares. It is standard practice, not a claim. Separately, $D_s{=}B{=}8$ makes four subspace codes a native 32-bit word that a packed scan loads once and unpacks in registers. After the layout fix the fetched sectors are already right, so packing reduces load instructions, addresses and loop iterations rather than logical bytes. We do not claim wider words are universally better; they add unpacking, dependencies and register pressure.

3.2 Cartesian factorisation of the primary table

The question is not how small the table can be but how much table state to materialise before saved residency is outweighed by added issued work. For subspace m and code c, let $T_m[c] = \|q'_m - C_m[c]\|_2^2$. Because $C_m[c]$ is a Cartesian product of one-dimensional levels and squared Euclidean distance is additive, any partition of the eight code decisions induces an exact sum of smaller tables. For the two-way partition, with $c_{\mathrm{hi}} = c \gg 4$ and $c_{\mathrm{lo}} = c \bmod 16$,

$$T_m[c] \;=\; T_m^{\mathrm{hi}}[c_{\mathrm{hi}}] + T_m^{\mathrm{lo}}[c_{\mathrm{lo}}]. \tag{1}$$

Each component holds 16 entries, so $G{=}2$ stores 32 per subspace and performs two lookups. Table construction falls by $16\times$, not the $8\times$ of resident entries: the full table evaluates $256D_s$ coordinate contributions per subspace and the split form $32(D_s/2) = 16D_s$. An equal G-way partition stores $G \cdot 2^{8/G}$ entries and needs G lookups per candidate–subspace, which states the trade-off analytically — state falls only until the table fits, while instruction demand keeps rising.

The residency target is narrower than the device's shared-memory limit, because the table is not the only claimant. The scan kernel's admission test is

$$\texttt{scan\_base} + M \cdot G \cdot 2^{8/G} \cdot 4 \;\le\; \texttt{smem\_optin}, \qquad \texttt{scan\_base} = 40 \cdot \texttt{BLOCK} \text{ bytes}, \tag{2}$$

with `scan_base` the per-thread candidate scratch claimed first. At the tested `smem_optin` = 101,376 B this leaves the table 94.0 KiB at `BLOCK`=128, falling to 59.0 KiB at `BLOCK`=1024. The full table is $M{\cdot}256{\cdot}4$ bytes — 96 KiB at $M{=}96$ and 384 KiB at $M{=}384$ — so *neither* clears the budget at any admissible block size, $M{=}96$ missing the most generous by 2 KiB, while both factorised tables clear the least generous at 12 and 48 KiB (Figure 1). The full table is thus off chip at both M, and what grows with M is the cost of the lookups that miss, since the scan pays one per candidate per subspace. The prediction we test in Section 6.4 is therefore directional: reducing table state should help until the extra lookup work outweighs the residency it buys, and the crossing should move with M and with probe depth together.

Section 5 states what grouped tables already owe to GPU IVF-RaBitQ. What Equation 1 adds is that the grouping exists at all here: it follows from the Cartesian construction, and does not exist for an ordinary k-means product quantiser at the same width. Exactness is in real arithmetic; it promises no bitwise-identical floating-point association or tie order.

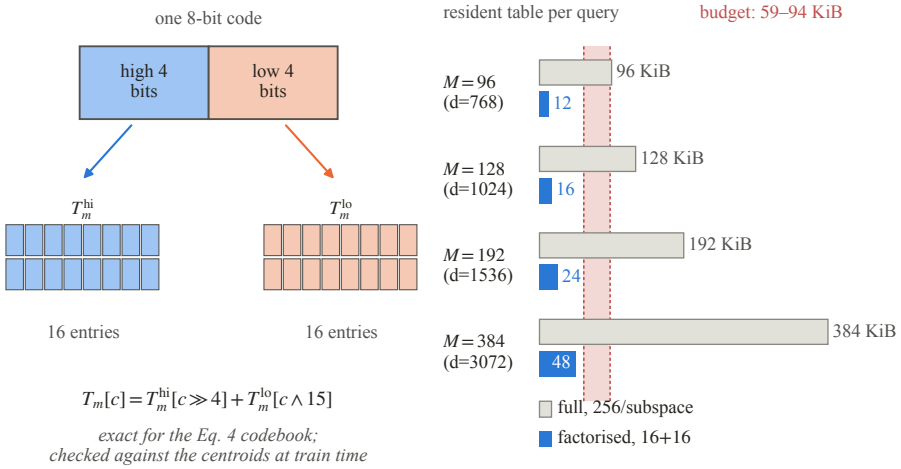

Fig. 1: Exact Cartesian factorisation of an 8-bit JHQ primary lookup table: 256 resident entries become two 16-entry tables. The band on the right is what Equation 2 leaves for the table across the admissible block sizes; every factorised table clears its lower edge and every full table overruns its upper one — M=96's by 2 KiB, the rest by more.

3.3 Selective residual refinement, and what it leaves open

Reading the per-dimension residual for every routed candidate would turn an $O(c_k d)$ stage into work closer to $O(n_c d)$, so only the c_k primary survivors trigger residual access. Fusing the two phases is a plausible alternative — it removes a round trip and a kernel boundary — and Section 6.5 reports why we reject it.

4 Adaptive Hierarchical Refinement

4.1 Criterion

Original JHQ leaves α externally specified, and the useful budget is workload dependent: at *nprobe*=128 openai3-3072 is flat from α=4 over the measured grid while arxiv-768 is still improving at α=200, a censored upper endpoint rather than a plateau. A single global constant therefore either wastes $O(\alpha k d)$ residual work on easy workloads or filters candidates before accurate refinement on hard ones. We calibrate a workload-level operating point rather than predict α per query, which keeps the control path label-free and amortisable.

Let Q_s be S sampled queries from the current workload and $R_{\max}(q)$ the top-k ID set returned under a generous $\alpha_{\max}$. For a tested α with returned set $R_\alpha(q)$, define the total slot disagreement

$$D(\alpha) \;=\; \sum_{q \in Q_s} \big[\, k - |R_\alpha(q) \cap R_{\max}(q)| \,\big], \tag{3}$$

and select the smallest tested-grid α with $D(\alpha) \le \epsilon$. The tolerance ϵ is a non-negative integer over the whole sample, not a per-query fraction. Set membership

asks whether more residual work changes *which* neighbours are returned while ignoring harmless permutations of the same IDs. S=32 and ϵ=1 were our initial defaults; Section 6.6 measures what they risk and withdraws the first of them. The two are also not independent: ϵ counts slots over the whole sample, so a fixed ϵ is a stricter test as S grows. Ground-truth recall is what we care about but is unavailable online, and a learned predictor costs labels, maintenance and drift behaviour outside this scope. Output stability is observable from the running index, but cannot recover routing losses, certify recall, or see past $\alpha_{\max}$; Section 6.6 validates it against ground truth offline rather than treating self-agreement as a guarantee.

4.2 Monotonicity and bisection

Proposition 1 (monotonicity of disagreement). *Let $\alpha_1 \le \alpha_2 \le \alpha_{\max}$. Under exact top-$c_k$ primary selection with c_k non-decreasing in α, a fixed refined distance, and a deterministic total tie rule, $D(\alpha_1) \ge D(\alpha_2) \ge D(\alpha_{\max}) = 0$.*

Proof. Exact top-c_k selection nests the survivor sets, $S_{\alpha_1} \subseteq S_{\alpha_2} \subseteq S_{\alpha_{\max}}$. Order $S_{\alpha_{\max}}$ by the refined distance with the deterministic tie key and let $R_{\max}$ be its first k elements. Any element of $R_{\max}$ present in S_α also appears in top-$k(S_\alpha)$, since fewer than k elements of $S_{\alpha_{\max}}$ precede it and hence fewer than k of the subset S_α do. So $R_\alpha \cap R_{\max} = S_\alpha \cap R_{\max}$, which can only grow with α. □

Implementation caveat and measured deviation. The production scan uses a bounded per-thread selector with `K_LOCAL`=4, so a thread can discard a globally top-c_k item when more than `K_LOCAL` of them fall in its stride class, and theorem-level nestedness is not asserted for the deployed kernel. We measure the gap rather than hide it behind aggregate recall: with the index pinned, vogue-768 at *nprobe*=128 and `BLOCK`=512, raising `K_LOCAL` from 4 to 8 changes 10 of 10,000 returned ID positions across 5 of 1,000 queries, and *no* query gets a different result set: every difference is two adjacent ranks swapping inside an identical top-10, so here the bounded selector reorders ties rather than discarding. Which items tie moves with the codebook, so this is one index instance, not a constant. `BLOCK`=512 is deliberately the harder collision regime, and `K_LOCAL`=8 is register-infeasible at the production `BLOCK`=1024.

The consequence for bisection should be stated precisely. Across all 15 calibration traces that probe two or more grid points, $D(\alpha)$ is non-increasing at the points the search visited. That is evidence, not a proof: points the search never visited are untested, so we do not claim bisection returns the global minimum acceptable α on the grid, only that its stopping behaviour is conservative with respect to the sampled criterion. Floating-point ties are a separate matter — the proposition assumes a deterministic total order, not bitwise-identical association.

Against the prior art of Section 5, our contribution here is narrow: a predictor-free top-k stability signal applied to JHQ's external residual budget, with its proxy error and economics quantified.

4.3 Calibration economics

With T_{cal} the calibration time and T_{fixed}, T_{rule} the per-batch times before and after selection, the break-even batch count is $B^* = T_{\text{cal}}/(T_{\text{fixed}} - T_{\text{rule}})$ whenever $T_{\text{rule}} < T_{\text{fixed}}$; Section 6.6 reports both terms. Reuse assumes the query distribution, routing configuration, k and index stay stable — the price of workload-level rather than per-query adaptation — and drift is unmeasured: recalibrating every H batches adds T_{cal}/H per batch, but choosing H is a drift-detection problem outside this evidence. The timer starts after sample copying, so the cost is algorithm-level.

5 Related Work

We organise related work around the novelty threats that matter for this paper rather than as a catalogue.

Quantisation and JHQ. Product Quantization [4] established Cartesian decomposition across subspaces and asymmetric distance computation; JHQ [3] adds an orthogonal transform and hierarchical residual refinement. We claim neither as a contribution: our work begins from that representation and asks how its specific structure should execute on a GPU.

Grouped lookup tables. This is the closest prior work and we state the overlap plainly. PQ Fast Scan [1] showed that cache residency alone is insufficient for fast ADC; Quicker ADC [2] generalised it with irregular product quantisers and split tables. Most directly, GPU IVF-RaBitQ [8] partitions dimensions into groups of U bits, builds one 2^U-entry table per group, sums D/U lookups per candidate, and selects U=4 by trading shared-memory capacity against arithmetic. That trade-off is the same one Section 3.2 navigates, it is prior art, and we do not claim it.

Two things are ours. First, the object: RaBitQ groups a representation designed to be grouped — its code *is* the bit pattern its table is indexed by — whereas JHQ's primary code is an 8-bit index into 256 codewords with no grouped structure on its face. Equation 1 is an identity that has to be derived from the codebook's Cartesian construction, and it does not exist for an ordinary k-means product quantiser at the same code width. Second, the method: RaBitQ selects its group size by measuring which performs best, while Section 6.4 reads an instruction count off the disassembled kernel and uses it to say which of the two competing costs each granularity is paying.

The two choices agree, and it is worth saying so rather than manufacturing a difference. RaBitQ's U counts bits per group and our G counts groups of an eight-bit code, so U=8/G and its U=4 is our G=2: both land on four bits and sixteen entries per table. Two derivations over different representations reaching the same group width is corroboration that the trade-off is real, not evidence that either is novel for choosing it.

Adaptive ANN control. ANN libraries such as FAISS [5] expose parameter sweeps, and learned approaches adapt search termination to the query [6]; we claim neither sampling nor parameter selection as new. Our rule differs in its signal: it needs no labelled neighbours and no learned predictor, comparing the system's own returned top-k sets against a generous-budget reference. Because a label-free proxy is not a recall guarantee, the evaluation quantifies its selection error and its amortisation separately.

GPU ANN systems. CAGRA [7] represents the graph-based GPU frontier and cuVS provides production implementations of it, IVF-PQ and IVF-RaBitQ [8]. These are not uniformly weaker than JHQ — our matched experiments show regime crossings in the high-recall tail and with batch size — which is why we report nondominated frontiers and configuration-specific build failures rather than a single aggregate speedup.

6 Evaluation

6.1 Setup and protocol

All GPU systems run on one NVIDIA RTX 5090. Two of its numbers are load bearing and appear again below: 170 SMs, and a 101,376-byte opt-in shared-memory limit per block, which is the budget the resident table competes for. The six datasets span 768 to 3,072 dimensions and 1.0M to 17.8M vectors; Figure 2 labels each with its shape.

We report Recall@10 against true top-10 ground truth; because c_k scales with αk, other k are not established. Timing runs host query input to host result output, and frontiers use steady-state throughput at batch 1,024 — the whole query set in one call — which Section 6.7 shows is a consequential choice. Occupancy statements are analytical resource-budget calculations: Nsight Compute counters were unavailable in the container (`ERR_NVGPUCTRPERM`), so we report none measured. Two recall noise floors are kept distinct: about 10^{-3} across builds in the documented cold-start experiment, and $\approx 10^{-4}$ for paired comparisons on one index from top-c_k tie breaking. The CPU comparison buckets recall at 10^{-3} so a tie-break-sized difference cannot buy a point a place; the GPU and baseline fronts are plain Pareto envelopes over the measured points, and the cuVS columns are the harness's three-run mean throughput, not a median. We do not enlarge batches by duplicating the roughly 1k available queries: duplication alone raises throughput (Table 2), so such a batch would measure an artefact.

Baselines are the cuVS implementations of IVF-RaBitQ [8], IVF-PQ, and CAGRA [7] in fp32 and int8. The frontier's comparison line is the IVF-routed family, which shares JHQ's routing structure, build-cost profile and interface; CAGRA is a graph index and is reported in Section 6.3 with its margins, its build cost and the query volume at which the two repay each other — separately, and in both variants, so that neither the one that wins nor the one that loses is the one we chose to show. JHQ runs use dense coarse training, `JHQ_N_TRAIN` $= 39 \cdot nlist$; we mix in no under-trained points and union no fronts

Table 1: Fastest system at each matched recall, batch 1,024, with its margin over JHQ. int8 and fp32 are cuVS CAGRA, RaBitQ is cuVS IVF-RaBitQ; "–" is a recall no two systems both reach. JHQ is fastest in 18 of 25 cells; the graph baseline is Section 6.3.

	fastest system at Recall@10					
dataset	0.90	0.93	0.95	0.97	0.98	0.99
vogue-768	**JHQ**	**JHQ**	**JHQ**	RaBitQ 1.0×	RaBitQ 1.1×	**JHQ**
arxiv-768	**JHQ**	**JHQ**	**JHQ**	RaBitQ 1.1×	RaBitQ 1.2×	–
bge-m3	**JHQ**	**JHQ**	–	–	–	–
stella-trec24	–	–	–	–	–	–
openai3-1536	**JHQ**	**JHQ**	**JHQ**	**JHQ**	**JHQ**	–
openai3-3072	**JHQ**	**JHQ**	**JHQ**	**JHQ**	RaBitQ 1.1×	RaBitQ 1.4×

across binaries. The CPU JHQ baseline runs the same index parameters on the same host with threads pinned (`OMP_PROC_BIND=close`, `OMP_PLACES=cores`) and its own (α, *nprobe*) sweep, sharing the training procedure but not a codebook, with one measurement per cell.

6.2 RQ1: where does JHQ actually lead?

Figure 2 gives the measured frontiers and Table 1 reduces them to a single question: at each matched recall, which system is fastest? Among IVF-routed methods JHQ owns 18 of the 24 comparable cells and IVF-RaBitQ the other 6, by 1.0–1.4×. The line drawn here is the index family: JHQ, IVF-RaBitQ and IVF-PQ share a routing structure, a build-cost profile and an interface, and the graph baseline is reported separately in Section 6.3 rather than dropped. On stella-trec24 no compared baseline builds at all, which is why its panel carries one curve. One further row is excluded throughout: stella-trec24's CAGRA-int8 point at Recall@10 0.9692 is `status=CONTAMINATED` in the harness's own record, and the frontier generator had dropped only `FAILED` rows.

Two things beside the table complete the picture. On bge-m3 and stella-trec24 the tested cuVS IVF-RaBitQ host-input build path fails with an out-of-memory allocation after entering streaming construction and CAGRA fp32 does not fit the card; on stella-trec24 IVF-PQ fails the same way, which is why Figure 2 shows it one curve short of bge-m3. JHQ indexes all six — a statement about the measured library and configuration paths on this device, not about those algorithms in general. And the throughput ordering has a build-time counterpart, reported in Section 6.9: JHQ trades search throughput below Recall@10=0.95 for a build several times cheaper, and for indexability at a code budget that is not smaller than int8's.

6.3 RQ1, continued: the graph baseline

CAGRA is off the frontier of Section 6.2 because it is a different index family, not because of how it scores. It is faster than JHQ where both run and the

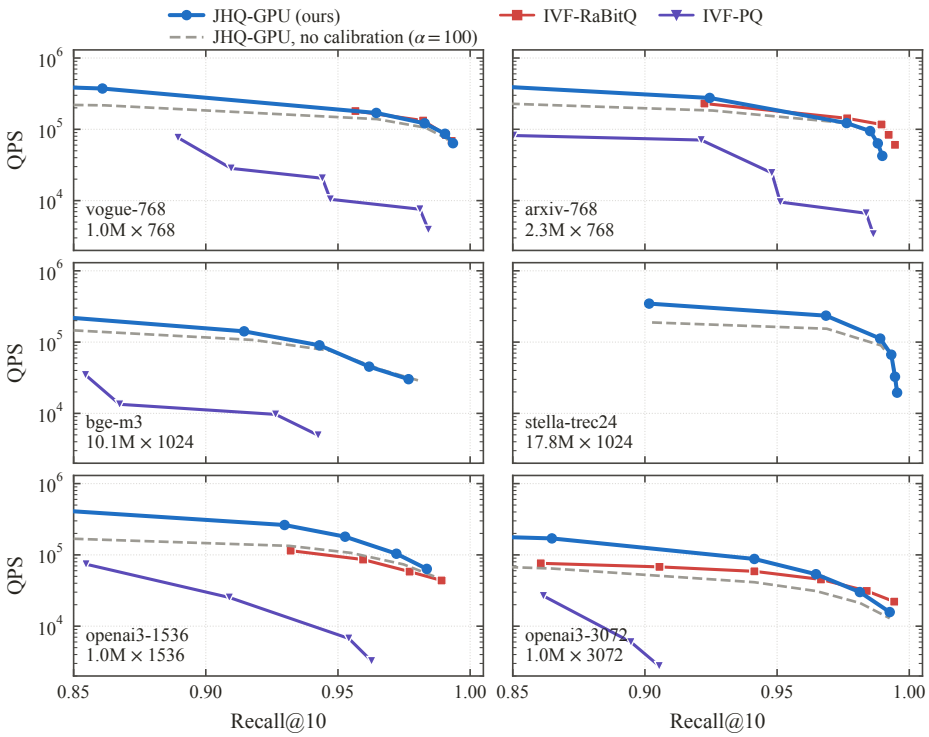

Fig. 2: Measured Recall@10–QPS frontiers, batch 1,024. Four datasets carry complete JHQ/IVF-RaBitQ curves; the two largest show the baselines that could be built and searched on the tested card. Sweep endpoints are measured endpoints, not architectural recall ceilings.

recall target is not high: CAGRA-int8 leads in 13 of the 15 cells they share, by 1.3–2.9×, and below Recall@10=0.95 in 10 of them by 1.6–2.9×; CAGRA-fp32 leads in 8 of 21 by 1.1–1.5×. We report this rather than scope it away.

What it costs is build time, and the trade is quantifiable rather than rhetorical. Figure 3 gives both variants beside JHQ, and where CAGRA int8 is the faster searcher its throughput repays the build gap only after 1.6M to 8.9M queries; below that volume JHQ costs less end to end. Neither CAGRA variant builds on stella-trec24 within the card, and fp32 does not build on bge-m3 either.

6.4 RQ2: does the design prediction hold?

Section 3.2 predicted that reducing table state helps only until extra lookups outweigh the residency saved. Figure 4 varies G with the represented distance fixed. If state alone governed throughput, G=4 and G=8 should win, since they store fewer entries than G=2. They do not: G=2 wins all eight configurations in Figure 4(a), and throughput falls monotonically as lookups per candidate rise. Its narrowest win is where the effect stops being resolvable — a second phase of the same run puts one of those cells on the other side of zero, and a single

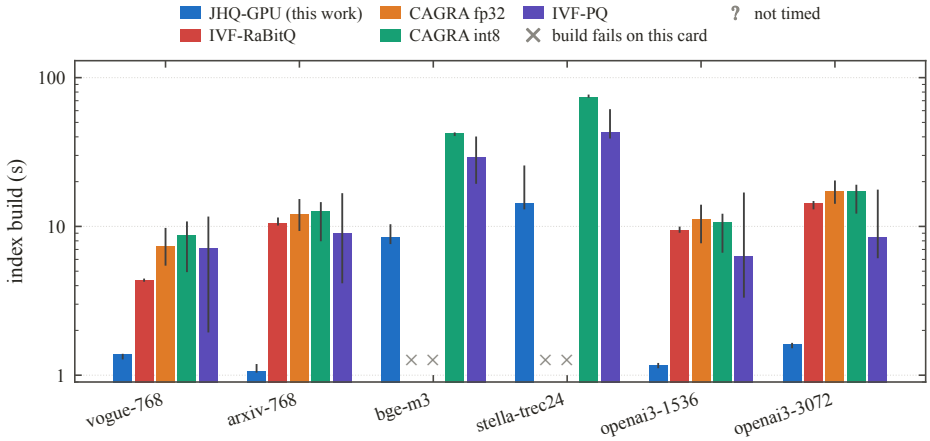

Fig. 3: Index build time, log scale. Bars are the median over swept configurations and repeat builds and whiskers their range, not a confidence interval. JHQ's bar is one cold training plus its median encode: training is cached after a dataset's first row and encoding is not. Each baseline's fastest build per dataset is used, which is the choice least favourable to us. The coarse-training budgets differ and are not controlled here: JHQ trains at 39 points a centroid, cuVS defaults to 256, so IVF-RaBitQ trains on 5.8 to 6.6× as many points. Part of its build gap is that budget rather than the index, and the same budget buys it better centroids at search time.

measurement a side cannot separate a 4% effect from run noise, which is why the design is stated as a regime rather than a rule.

The sharpest form of the argument is internal to the losers. $G \cdot 2^{8/G}$ is 16 for both G=4 and G=8, so the pair holds the table footprint fixed and differs only in the number of lookups and the address arithmetic they need. Residency is constant, issued work rises, and throughput falls with it — by 5.8 to 38.3%, widening with probe depth. That pair, rather than the table-free control, separates the two effects most cleanly.

The regime change with M appears as predicted, and it is a scaling property rather than a constant: factorisation is worth a few percent at M=96 and up to +53.8% at M=384, rising with probe depth at the larger M. The mechanism is the same one throughout — a table that misses is paid once per candidate per subspace, so its cost grows with M and with the candidates scanned together. At M=96 the gain neither rises reliably nor exceeds the noise, which is the same statement as the ambiguous cell above: the full table misses the carveout by 2 KiB there against 290 KiB at M=384, so there is almost no residency to win back.

6.5 RQ3: can the byte-count explanation be rejected?

Table 2 collects the controls that vary bytes and issued work independently; its third column names the version of "fewer bytes is faster" each row rejects.

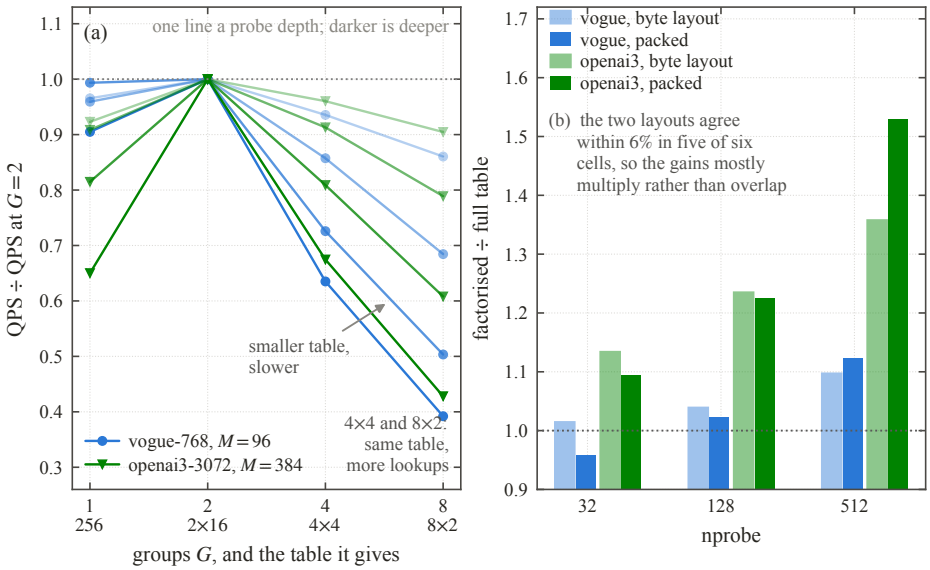

Fig. 4: (a) G=2 wins all eight configurations of the granularity sweep although G=4 and G=8 store fewer entries and occupy the same footprint as each other; every line is one probe depth, normalised to its own G=2. (b) Factorisation matters far more at M=384 than M=96 and stays largely complementary to packed loading. The two panels are different phases of the run — (a) the granularity sweep at *nprobe* $\in \{8, 32, 128, 512\}$, (b) the two-by-two table×layout phase at *nprobe* $\in \{32, 128, 512\}$ — and are counted separately.

Table 2: Controls that separate bytes moved from work issued. Every row holds the represented distance fixed.

change	effect on throughput	what it rules out
table-free exact distance	−30 to −52%	least state is fastest
G=8 against G=4	−5.8 to −38.3%	same footprint, same speed
packed 32-bit loads	+1 to +66%	bytes moved predict speed
private probe cursor	−1.4 to +148.5%	loop work is negligible
fused residual refinement	−14.2%, −14.8%	fewer kernels is better
query duplication D=8	+4 to +26%	reuse alone drives the scan

Two are worth reading twice. Removing almost all table state loses heavily while *raising* the resident-block count the shared-memory budget implies, so neither residency nor attainable occupancy alone is the bottleneck. And moving identical bytes in 32-bit words gains up to 66%, rising with probe depth for the reason the factorisation gain does: a load is paid once per candidate per subspace. Table construction is under 3% of a batch, so faster table building does not explain the factorisation either.

The controls reject competing explanations but do not measure the variable they implicate, and static instruction counts need no profiler for that. `cuobjdump`

`-sass` on `scan_ivf_coalesced_kernel` finds exactly G generic loads and $G{+}1$ floating-point adds per unrolled subspace iteration — the second is the check that the normalisation is right — giving 5.0, 13.0, 25.0 and 50.4 static instructions per candidate–subspace at $G{=}1, 2, 4, 8$. One statement then needs no model: $G{=}1$ *issues* 5.0 *against* $G{=}2$*'s* 13.0*, and is slower at most operating points anyway.* It issues the fewest of any granularity while holding the most table state — 256 entries a subspace against 32 — so instruction count predicts it should be fastest, it is not, and the state it holds is the one thing that separates it. Extending a fit over the three resident granularities back to $G{=}1$ sizes that shortfall and shows it growing with probe depth and with M, but the size is an extrapolation beyond the fitted range, so we read its sign and trend only. Instruction counts and controlled comparisons are together consistent with this account on the tested card; they are not an independent measurement of the two terms, which would need a build holding residency fixed while sweeping instruction count alone.

6.6 RQ4: would one fixed budget do instead?

Five arms share one index and one timing harness, calibrating on even-indexed queries and reporting recall on the odd-indexed half: each fixed α on the grid; the sampled rule over 64 random samples at each S; the same criterion on the whole calibration pool; a whole-grid walk on the sample the rule bisects; and, for reference only, the fastest α within 10^{-3} of the best any α reaches, chosen with ground truth. Table 3 is the result.

A constant meets the quality bar only by overpaying. The reference budget is an order of magnitude apart on the two datasets; its conservative end generalises and its cheap end does not, so the only fixed choice safe everywhere is the expensive one, and on openai3-3072 that costs 32% of throughput for no recall. The question is not whether a constant exists but what it costs.

The rule finds it, given enough sample. Its median selection matches the reference wherever the reference is resolved, and where it does not — $S{=}32$ on vogue-768 — it errs cheap, not expensive. Against the $\alpha{=}100$ default the selection is worth 1.095–1.469× in steady state.

The sample is the risk, and $S{=}32$ *is too small.* The right-hand block of Table 3 is a draw-level failure rate, not an average: at $S{=}32$ most draws on vogue-768 give up more than 10^{-3} of held-out recall, and at $S{=}128$ that is at worst 8% — small, and not zero. A 2,000-draw resampling shows the rate is workload dependent rather than a property of S. We withdraw Section 4's $S{=}32$: S buys tolerance, and $S{=}128$ is where the tolerance becomes small.

Neither the sample nor the bisection is the approximation. The whole-grid walk reproduces the bisection's choice in all 768 draws, so the search is exact on what it sees; evaluating the criterion on the entire pool costs more and selects a *more* conservative budget. The reason is in the criterion: ϵ counts disagreeing slots absolutely while a sample of S queries has $S{\cdot}k$ of them, so the same $\epsilon{=}1$ tightens as the sample grows. ϵ and S cannot be chosen independently, which is the rule's substantive limitation.

Table 3: The five arms on one index. *ref.* is the ground-truth reference budget; the rule's median selection over 64 samples is shown at each S, then the share of those samples that give up more than 10^{-3} of held-out recall, then what the criterion selects on the whole calibration pool. On openai3-3072 every α from 4 to 16 is within 10^{-3} of attainable recall, so the reference there is resolved only to its magnitude.

		rule's α at S			% draws losing $> 10^{-3}$		full
dataset / *np*	ref.	32	64	128	32	64	128	pool
vogue / 128	64	32	64	64	69	45	8	100
vogue / 512	64	32	64	64	59	36	3	100
openai3 / 128	8	4	4	4	25	6	0	4
openai3 / 512	8	4	4	4	30	2	0	4

Table 4: Matched-recall JHQ / IVF-RaBitQ throughput, by batch size, at Recall@10 of 0.90 and 0.95, on the four datasets IVF-RaBitQ builds on. Each system's curve is interpolated at the target recall; "–" is a recall the two curves do not both reach.

	vogue-768		arxiv-768		openai3-1536		openai3-3072	
batch	.90	.95	.90	.95	.90	.95	.90	.95
32	0.68	0.55	–	0.93	1.85	1.08	2.23	1.34
128	0.99	0.92	–	1.47	2.75	1.86	3.19	2.14
512	1.06	1.01	–	3.90	2.56	1.91	2.30	1.51
1024	1.09	1.33	–	1.18	2.40	1.93	1.89	1.41

Two earlier observations survive. An earlier prototype of the rule recovers the swept α in three of four configurations at *nprobe*=128, the fourth censored by $\alpha_{\max}$; and over a broader 32-configuration sweep against α=100 — six datasets at S=32, a different configuration from this section's, which we do not combine with it — gain is 1.044–2.653× for at most 0.0048 recall.

6.7 RQ5: does batch size change the ordering?

Table 4 gives matched-recall ratios against IVF-RaBitQ at four batch sizes, on the four datasets where IVF-RaBitQ builds. It does: JHQ leads at every measured point on three of them, while on vogue-768 it crosses parity between batch 128 and 512. The frontier ratios of Section 6.2 are therefore the batch-1,024 column of this sweep and not batch-independent summaries. One cell is absent, arxiv-768 at R=0.90, where the two curves do not overlap at that recall.

6.8 RQ6: what does the port buy over CPU JHQ?

Figure 5(a) puts both sides' (α, *nprobe*) envelopes on one recall axis. Across the four datasets the CPU baseline covers, the GPU reaches 52 to 108× the CPU

envelope; pinning α=100 on *both* sides, so neither may tune the budget, gives 41 to 114×. We give both denominators because they move in opposite directions: freedom over α and threads is worth more to the CPU on some datasets than on others. Four of six, because the reference is the authors' own artifact rather than a reimplementation and bge-m3 and stella-trec24 were not reached; the comparison is therefore not made on the two datasets where JHQ is the only system that builds.

Panel (b) is why this comparison is worth a figure. The same α knob is worth 2.5 to 3.0× more on the GPU than on the CPU, at every probe depth. Selective refinement is therefore not a CPU idea carried across: on the CPU the scan dominates and the budget barely matters, while on the GPU the scan is fast enough that refinement becomes the term worth choosing — which is what Section 6.6's rule exists to choose.

Four limits apply. The GPU side of the vogue comparison is the rule's chosen α at each *nprobe*, not an exhaustive sweep. CPU rows below *nprobe*=128 are dropped, because 1,000 queries finish fast enough there that thread start-up dominates the timer and throughput rises with *nprobe*; the recall column is unaffected. The top vogue point rests on the noisiest cell we measured — one α there swings a quarter of its own value across five repeats while its neighbour swings 2% — so we quote medians, and the single-pass log had given 90× where the median gives 85. And every other cell is measured once, so these are order-of-magnitude figures; the envelope keeps the fastest point per recall bucket, which biases the CPU side upward and so does not inflate the ratio, but we claim no statistical bound.

6.9 Construction and memory

Figure 3 gives cold build times on every dataset. JHQ is the cheapest to build on all six and the gap widens with N: at stella-trec24 it is 13.4 s against 74.3 s for CAGRA int8. Accounting must respect caching: training can be cached while add is not, so taking maxima independently fabricates a time that never occurred.

On memory we claim no advantage: in the four datasets with measured VRAM JHQ uses 22–40% more than IVF-RaBitQ, and its budget is not smaller than int8's. JHQ occupies a different memory–accuracy–throughput point and still indexes the two largest datasets where the compared paths fail. The accounting carries an unattributed component that should not be relabelled as allocator overhead without direct measurement.

7 Conclusion

We asked how JHQ's primary representation should execute on a GPU and how much of its hierarchy is worth executing. For the first, the Cartesian codebook admits an exact two-table factorisation whose state grows as $32M$ against $256M$, and the payoff rises with both M and probe depth while still-smaller splits lose to their extra lookups. Disassembling the kernel adds the term the controls could

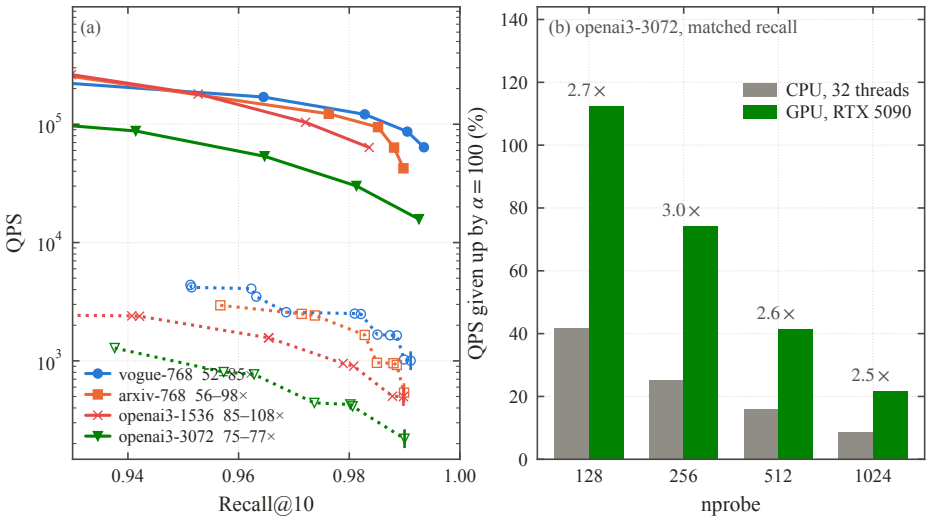

Fig. 5: GPU against CPU JHQ at the same index parameters, on the four datasets the CPU reference covers. (a) Both sides' $(\alpha, nprobe)$ envelopes: solid is the GPU, dotted with open markers the CPU points the envelope is taken over, and the legend gives each dataset's range of matched-recall ratios. (b) What the α budget is worth on each processor, at matched recall on openai3-3072 — the reason the budget is worth choosing on a GPU and not on a CPU.

not reach: the granularity whose table does not fit issues the fewest instructions per candidate–subspace and is slower anyway, so the target is a residency budget rather than minimum bytes. For the second, a label-free stability rule buys 1.095–1.469× at S=128 and selects the offline reference budget wherever that reference is resolved — but some samples still exceed a 10^{-3} loss, so the sample size is a risk decision, not a default. The region this defines is bounded: among quantised IVF methods JHQ is fastest in 18 of the 24 cells it shares with the other IVF-routed methods, while the graph baseline is faster below Recall@10=0.95 at several times the build cost. Remaining gaps are k beyond 10, workload drift, and cross-GPU validation.

References

1. André, F., Kermarrec, A.M., Scouarnec, N.L.: Cache locality is not enough: High-performance nearest neighbor search with product quantization fast scan. Proc. VLDB Endow. **9**(4), 288–299 (2015). https://doi.org/10.14778/2856318.2856324
2. André, F., Kermarrec, A.M., Scouarnec, N.L.: Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. IEEE Trans. Pattern Anal. Mach. Intell. **43**(5), 1666–1677 (2021). https://doi.org/10.1109/TPAMI.2019.2952606
3. Han, J., Zhang, M., Trajcevski, G.: JHQ: Johnson-Lindenstrauss enhanced hierarchical quantization for high-dimensional approximate nearest neighbor search. Proc. VLDB Endow. **19**(7), 1530–1543 (2026). https://doi.org/10.14778/3801059.3801067

4. Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. IEEE Trans. Pattern Anal. Mach. Intell. **33**(1), 117–128 (2011). https://doi.org/10.1109/TPAMI.2010.57
5. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. arXiv:1702.08734 (2017)
6. Li, C., Zhang, M., Andersen, D.G., He, Y.: Improving approximate nearest neighbor search through learned adaptive early termination. In: SIGMOD. pp. 2539–2554 (2020). https://doi.org/10.1145/3318464.3380600
7. Ootomo, H., Naruse, A., Nolet, C., Wang, R., Feher, T., Wang, Y.: CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs. In: IEEE ICDE (2024). https://doi.org/10.1109/ICDE60146.2024.00323
8. Shi, J., Gao, J., Xia, J., Feher, T.B., Long, C.: GPU-native approximate nearest neighbor search with IVF-RaBitQ: Fast index build and search. Proc. VLDB Endow. **19**(11), 3454–3467 (2026). https://doi.org/10.14778/3836663.3836701